Bro, before writing even a single line of code, let's understand what the problem statement actually means, what the transport department expects from us, and what challenges our system must solve. We'll break it down like we're analyzing a real-world engineering problem.

1. Understanding the case study



Imagine a fictional town called Vetrinagar. It operates a small public bus service with a depot, several bus stops, and a limited number of buses.

During peak hours, passenger demand increases at certain stops. The transport supervisor receives passenger-demand information and manually decides which buses should be sent where.

For example:

- Railway Station has 35 passengers waiting.
- Market has 12 passengers waiting.
- College has 28 passengers waiting.
- Only three buses are available, each with a fixed seat capacity.
- Some roads are slow because of traffic.
- Another bus may break down before being assigned to a trip.

The supervisor must decide which passengers to prioritize, which bus to assign, and which route to take.

That is the problem our application will simulate.

2. What is wrong with the existing manual system?

The case study identifies four main problems.

Problem 1: High-demand stops are not prioritized

A stop with 35 waiting passengers might receive the same attention as a stop with just 5 passengers.

Our solution: PriorityQueue

Rank demand batches by passenger count, with earlier arrivals winning ties.

Problem 2: Passenger demand exceeds bus capacity

Suppose 32 passengers need a ride, but the assigned bus has only 25 seats. Assigning all 32 would violate the capacity constraint.

Our solution: Capacity-aware allocation

Board no more passengers than the available seats. Preserve the unserved passengers as a pending batch, maintaining its original arrival order.

Problem 3: Traffic makes selected routes inefficient

The shortest road by distance is not necessarily the fastest road. A 4 km road

with heavy traffic could take longer than a 6 km alternative.

Our solution: Dijkstra's algorithm using travel time as the edge weight

The system calculates the minimum-time route instead of simply choosing the shortest-distance route.



Problem 4: A bus becomes unavailable

A bus might break down before starting its assigned trip. Its passengers still need service, and its assigned route may need to be recalculated.

Our solution: Dynamic reassignment

Reassign only waiting or not-yet-started services. A trip that has already started must not be reassigned under the project's rules.

3. Breaking down the main problem statement

The central question is:

> How can we design and implement a Java-based system that prioritizes high-demand stops, allocates available buses within capacity, and updates feasible relief routes as demand and road conditions change?

This sentence contains three separate engineering problems.

Part A Demand prioritization
Decide which waiting passenger group should receive service first.
Inputs: passenger count and arrival order.
Output: a ranked queue of waiting demand.
Part B Bus allocation
Decide which available bus can serve the selected demand without exceeding its capacity.
Inputs: available buses, seat capacities, passenger counts and stop reachability.
Output: feasible assignments and any unserved demand.
Part C Route optimization
Calculate the fastest route from the bus's current location to the pickup stop and then to the requested destination.
Inputs: road network and travel times.
Output: the selected route and its estimated travel time.

These three parts must work together. A bus assignment is not valid merely

because a bus has enough seats; the required stops must also be reachable, and the bus must be available.

4. Understanding relief bus allocation

A relief trip is a special service sent to collect waiting passengers and take them to a requested destination.

For instance, consider this illustrative scenario.





Available bus: MiniBus-01

25 seats, currently available

ALLOCATION RESULT

25

Passengers boarded

7

Passengers pending

The seven unserved passengers remain in the system. They must not disappear from the queue or be silently discarded.

Notice an important distinction: passenger priority and passenger grouping are not the same thing.

- The `PriorityQueue` ranks demand based on passenger count and arrival order.
- The `ArrayDeque` preserves FIFO order among batches belonging to the same pickup-stop and destination group.
- The allocation logic must respect both the global priority rule and the group-level FIFO rule.

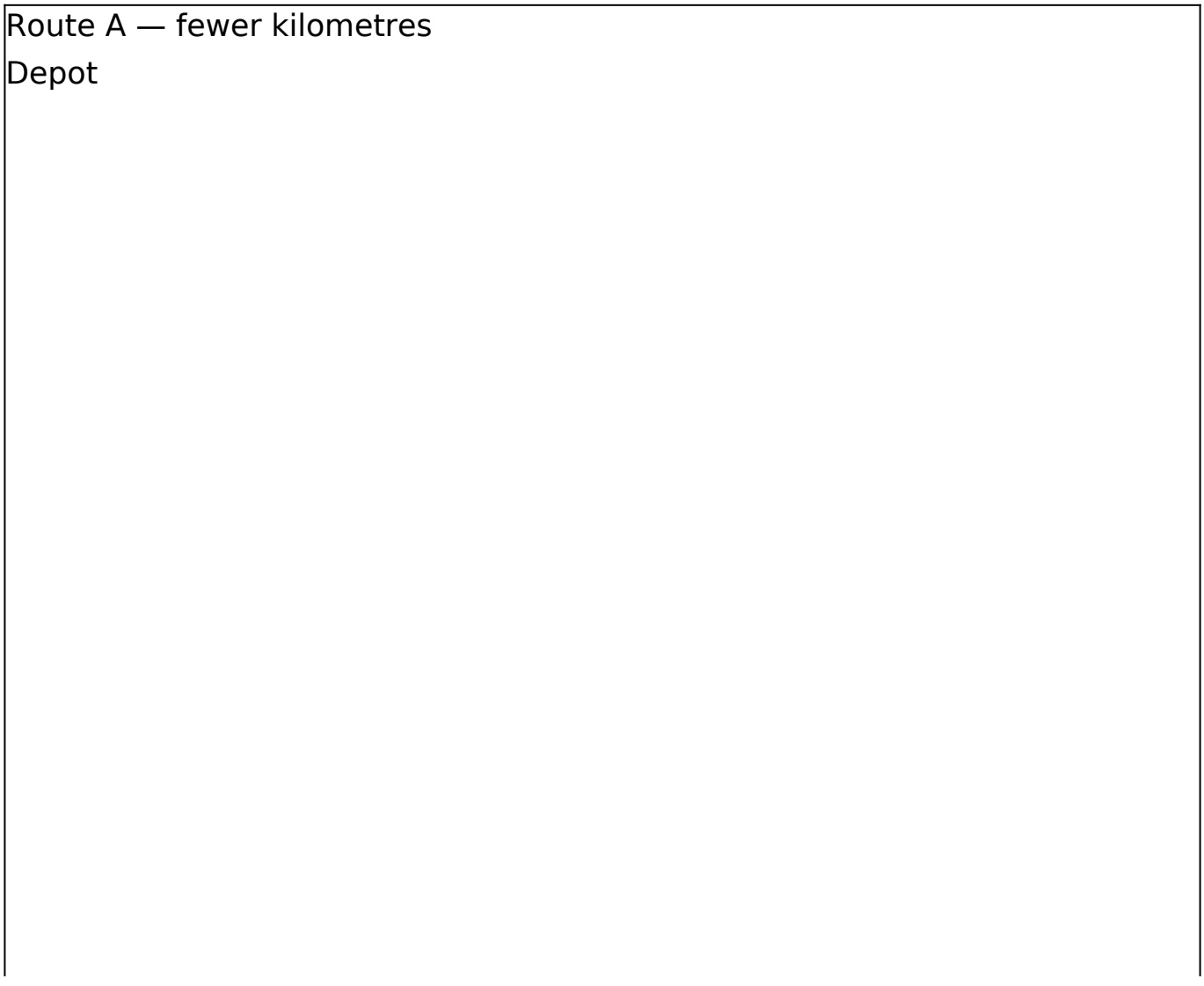
We will need to implement these rules carefully so that splitting a batch across multiple buses doesn't lose its identity or original arrival information.

5. Understanding travel-time-based routing

Our town can be represented as a weighted graph.

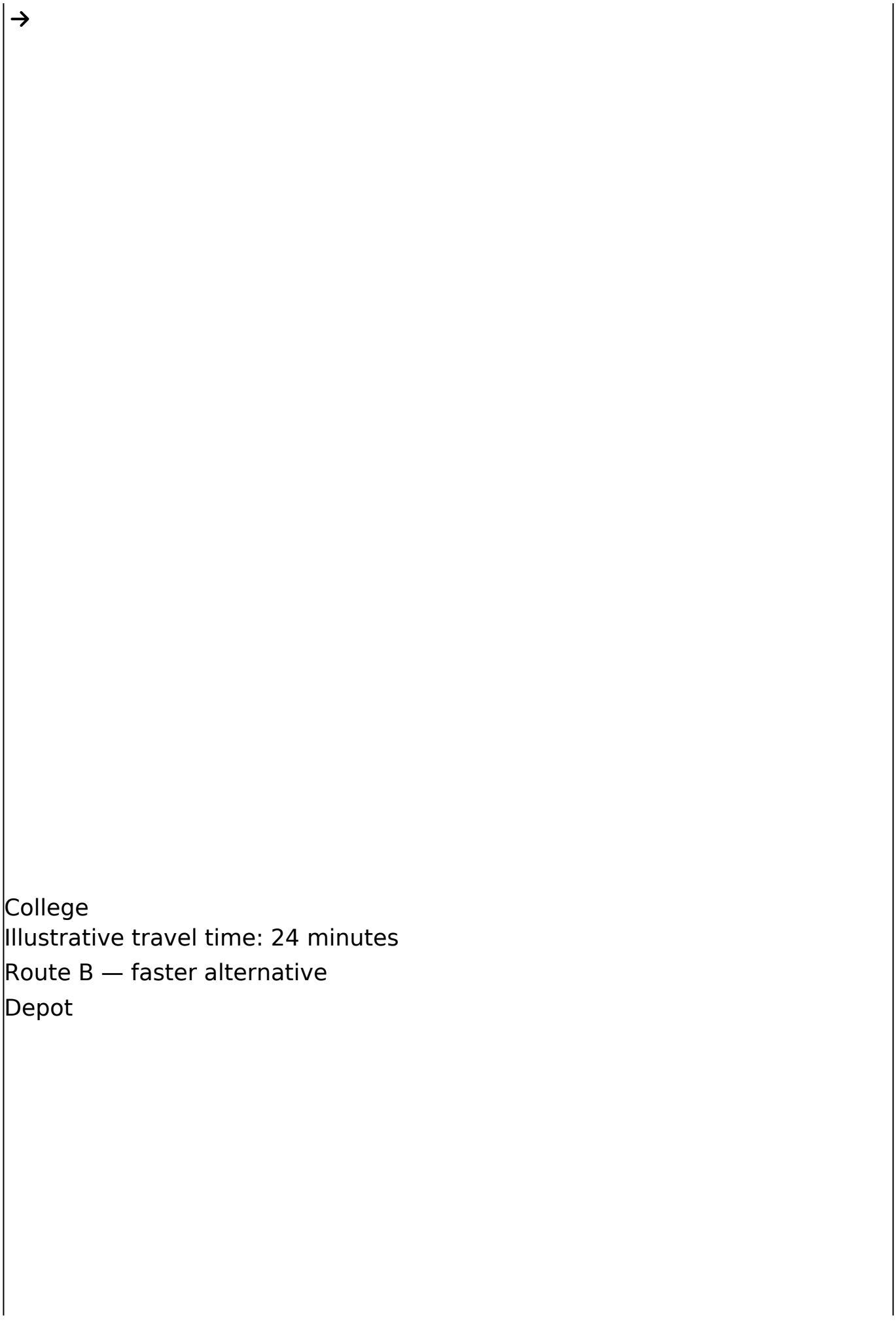
- Vertices represent bus stops and the depot.
- Edges represent roads connecting those locations.
- Edge weights represent travel time, such as minutes.

Consider these two possible routes from the depot to a pickup stop.



→

Market


College
Illustrative travel time: 24 minutes
Route B — faster alternative
Depot

→

Hospital



Dijkstra's algorithm should select Route B because its total travel time is lower.

If traffic increases the time on one road, the graph's weight can be updated and the route recalculated. If a road becomes disconnected, the algorithm must identify that the destination is unreachable rather than inventing a route.

One further detail: our system must optimize both route segments for each relief trip:

1. Bus's current location → pickup stop.
2. Pickup stop → requested destination.

The two segments must both be feasible before the assignment is accepted.

6. What the project scope allows—and what it does not

In scope	Out of scope
10–15 stops and one depot	Modelling an entire real city
3–5 buses with fixed seat capacities	Unlimited bus availability
At least 20 manually entered or synthetic demand batches	Live passenger sensors
Positive road travel times	Live traffic APIs
Demand prioritization and capacity-aware boarding	Global optimization of a full city timetable
Reassignment of eligible trips	Reassigning services already in progress
Console menu and documented tests	A mandatory GUI or database

The project is a simulation of operational decision-making, not a complete production public-transport management system.

7. Java OOP concepts: why each is required

The specification also evaluates how we structure the software, not just whether our output is correct.

Concept	Role in our project
Classes and objects	Model buses, stops, demand requests, relief trips and the road network.
Encapsulation	Prevent invalid seat occupancy and bus-status changes.
Inheritance	Define `MiniBus` and `StandardBus` as specialized bus types.
Polymorphism	Allow bus subclasses to implement `canUseRoad()` differently.
Abstraction	Define demand-ranking policies through `DemandPolicy`.

Composition	Let `TransportService` coordinate buses, demand records and the route planner.

The goal is to keep responsibilities separate. For example, the bus object should enforce its own seating limits, while the route planner should calculate paths. Neither should need to contain the entire application's menu logic.

8. The hidden challenges we should pay attention to

These are the tricky details likely to distinguish a reliable implementation from a basic one.

1. Deterministic priority: If two batches have the same passenger count, the earlier arrival must come first. The program must produce the same result for the same input.
2. Changing priorities: If a batch's ranking fields change, its existing priority-queue record must be removed and reinserted. Mutating an object in place is not enough to update its position in a Java `PriorityQueue`.
3. Partial boarding: When a bus cannot take an entire batch, the remaining passengers must retain their original arrival order and destination.
4. No reachable route: A bus with enough seats cannot be assigned if it cannot reach the pickup stop or destination.
5. Busy versus unavailable: A bus serving a trip is not the same as a broken-down bus. The program must distinguish these states.
6. Changing conditions: A traffic update or breakdown should affect eligible future assignments without rewriting trips that have already started.

Our understanding of the problem

At its core, this project is about coordinating three limited resources or constraints: passenger demand, bus capacity, and road travel time. The application must balance them using the specified algorithms and data structures, while preserving passengers' requests when the available resources are insufficient.

Before we begin the architecture, there's one design detail worth clarifying: the specification requires FIFO ordering within each pickup–destination group and global ranking by passenger count, then arrival order. We'll need to ensure these two rules coexist without allowing a later batch in the same group to overtake an earlier one. That should be an explicit allocation rule rather than an accidental consequence of how the queues are implemented.